Distributed Raft KV Store: Experiments and Results

CS 6650 Building Scalable Distributed Systems, Northeastern University

Shreyans Mulkutkar Pritam Mane Kevin Johnson

April 2026

1 System Overview

We built a distributed key-value store in Go combining Raft consensus with consistent-hashing-based sharding. Each shard is a 3-node Raft cluster (1 leader + 2 followers). Writes go through the Raft leader, which replicates to a majority before returning HTTP 200. Reads are served directly from the leader's in-memory map. Persistence uses a write-ahead log (WAL) and periodic JSON snapshots (every 100 writes). A single-node *SimpleKVS* baseline exposes the same HTTP API without Raft.

All four experiments ran on AWS EC2 (`t3.micro`, `us-east-1`) using Locust for load generation and Terraform for infrastructure as code. All Raft nodes run from a single Docker image (`pritammane105/raft-kv:latest`).

2 Experiment 1: Replication Overhead

2.1 Purpose and Tradeoff

Question: How much latency and throughput cost does Raft consensus replication add over a single-node store under write-heavy load?

Tradeoff: Consistency and fault tolerance vs. write performance. Raft requires at least one network round-trip per write before acknowledging the client. SimpleKVS flushes the WAL synchronously to local disk before responding.

Setup: 50 concurrent Locust users, 75% PUT / 25% GET, random keys from a 10,000-key space, ≈90 seconds per run, all nodes in the same Availability Zone to minimise network variance.

Limitation: Raft and SimpleKVS differ not only in replication but also in WAL flush strategy Raft flushes asynchronously while SimpleKVS flushes synchronously. This confounds a pure replication-cost measurement.

2.2 Results

Metric	SimpleKVS	Raft (3-node)
Total RPS	540	**585**
PUT avg latency	67.6 ms	**57.5 ms**
PUT p50	60 ms	**52 ms**
PUT p95	110 ms	**97 ms**
PUT max	650 ms	1200 ms
GET avg latency	38.2 ms	38.7 ms
GET failures	299 (404)	0

Table 1: Replication overhead results (50 users, same AZ). Full Locust reports in `replication_overhead/results/`.

2.3 Analysis

Raft achieved *higher* throughput and *lower* average write latency than SimpleKVS. The explanation is architectural: SimpleKVS flushes the WAL synchronously on the HTTP handler goroutine before returning the response. The Raft node sends HTTP 200 as soon as the majority acknowledges the write *in-memory*; WAL and snapshot flushes happen asynchronously in `RunApplyLoop` after the response is already sent.

Raft therefore trades a small durability window (a simultaneous crash of all three nodes before the apply loop flushes could lose the last few committed writes) for lower observed latency. The real cost of consensus appears only in the tail: Raft max PUT latency (1200 ms) is nearly double SimpleKVS (650 ms), reflecting occasional consensus coordination delays near the heartbeat timeout boundary.

GET latency is identical for both systems (≈37–39 ms at p50) because reads bypass Raft and are served from the in-memory map. The 299 SimpleKVS GET failures are HTTP 404s caused by reads arriving before the key space was populated not server errors.

Conclusion: Raft replication overhead in same-AZ deployments is negligible at this scale. The async apply loop dominates latency, not consensus itself. The trade-off cost appears in tail latency, not in the average.

3 Experiment 2: Horizontal Scaling

3.1 Purpose and Tradeoff

Question: Does write throughput scale linearly when doubling from one shard (3 nodes) to two shards (6 nodes)?

Tradeoff: Operational complexity vs. throughput. Adding a second shard doubles hardware and introduces a routing layer (FNV-1a consistent hashing), but each shard maintains an independent write quorum so throughput should scale near-linearly.

Setup: Four Locust runs at 75% PUT / 25% GET, random keys from a 10,000-key space. The client router hashes each key using the same FNV-1a function as `shard/hashing.go`. A 2.5-minute extended run tested sustained behaviour under load.

Limitation: 25% of traffic is reads, which do not benefit from sharding since all reads are served from the in-memory map regardless of shard count. Clients ran from a single laptop, limiting total concurrency measurement.

3.2 Results

Metric	1s/50u	1s/100u	2s/100u
Total RPS	565	555	**945**
PUT avg (ms)	61	180	40–123
PUT p50 (ms)	54	170	**38**
PUT p95 (ms)	100	380	**50**
PUT max (ms)	798	3500	1562
PUT failures	0	0	0

Table 2: Horizontal scaling results. 1s/2s = shards, u = users. p50/p95 for 2-shard show the lower-latency shard values. Full reports in `horizontal_scaling/results/`.

CloudWatch CPU utilisation peaked at 12.6% during the heaviest test. During the 2.5-minute extended 2-shard run, shard1 experienced 7,450 consecutive HTTP 503 responses for ≈1–2 seconds before self-healing. Response time spiked to 2,500 ms (p95) then returned to ≈50 ms.

3.3 Analysis

Total RPS increased from 555 (1-shard, 100 users) to 945 (2-shard, 100 users) a **1.7× improvement**. The theoretical maximum is 2×; the gap is explained by 25% read traffic, Locust coordination overhead, and imperfect 50/50 hash distribution.

The 1-shard tests reveal a clear saturation boundary: 50 users → 61 ms avg PUT and 565 RPS; 100 users → 180 ms avg PUT and 555 RPS. Throughput plateaued while latency tripled the signature of Little's Law saturation. CPU at 12.6% confirms the bottleneck is **disk I/O**, not compute. The culprit is the synchronous snapshot flush in `RunApplyLoop` every 100 writes: at 420 writes/sec this fires every ≈240 ms, serialising the full KV map to JSON on disk.

Two shards keep each leader below the saturation point (≈50 users each). The asymmetry between shard0 (85 ms p50) and shard1 (38 ms p50) in the 2-shard test confirms that **snapshot file size directly impacts write latency**: shard0 had accumulated data from the prior 1-shard run, making each flush more expensive.

The 503 burst during the extended test is a live Raft re-election event. The shard1 leader missed heartbeats (likely blocked on disk I/O), followers timed out, a new leader was elected in ≈300–600 ms, and the cluster refused writes during that window. This is Raft's safety guarantee working correctly: consistency over availability during elections.

Conclusion: Sharding delivers near-linear write throughput scaling. The primary bottleneck is disk I/O from snapshot flushes, not Raft consensus overhead. Sharding distributes this bottleneck by giving each shard an independent flush cycle.

4 Experiment 3: Leader Election and Failover

4.1 Purpose and Tradeoff

Question: Does the cluster automatically re-elect a leader under live write load, and are committed writes preserved with zero data loss?

Tradeoff: Availability vs. consistency during failure. Raft refuses writes during the election window rather than risk serving stale data. The cost is a brief outage; the benefit is that no committed write is ever lost.

Setup: Three `t3.micro` nodes in `us-east-1`. Locust ran 20 users (75% PUT / 25% GET) at 183 RPS baseline. Leaders were stopped mid-load via `aws ec2 stop-instances`. The experiment script wrote 10 keys before each crash, measured re-election time, verified data survival on the new leader, wrote 10 more keys post-crash, and restarted the crashed node to verify log catchup. Locust ran simultaneously in a separate terminal.

Limitation: `aws ec2 stop-instances` takes 15–30 seconds to fully halt an instance. During shutdown the node continued sending heartbeats, delaying quorum loss and inflating re-election times compared to an SSH process kill, which was used in the March 28 election-only experiment (190–381 ms).

4.2 Results

Metric	Value
Re-election time (min)	641 ms
Re-election time (max)	1739 ms
Re-election time (avg)	≈1200 ms
Total data loss	**0 keys**
Pre-crash writes survived	**100%**
Locust RPS baseline	183 RPS
Locust latency at crash	spike to 8000 ms

Table 3: Leader election failover results (3 rounds, 2 runs). Locust graph available at `leader_election/results/leader_election_failover.png`.

Figure 1: Locust throughput (RPS) and response time during leader crash and re-election. RPS drops to zero at the crash moment, latency spikes to 8000 ms, then both recover after re-election completes.

4.3 Analysis

Zero data loss across all successful rounds confirms the core Raft safety property: HTTP 200 is returned only after a majority of nodes have appended the entry to their in-memory log. Even if the leader then crashes, the entry exists on at least one follower and will be applied to the store after re-election.

Re-election times (641–1739 ms) are longer than the previous election-only experiment (190–381 ms) due to the EC2 instance stop mechanism (15–30 seconds) vs. an instant process kill. During the shutdown window the stopping node kept sending heartbeats, delaying follower timeout.

The Locust graph shows the failover cycle directly: steady 183 RPS → drop to near zero → latency spike to 8000 ms → recovery to 180 RPS. The V-shape (not a cliff) reflects Locust's 3-second request timeout failing fast rather than hanging indefinitely. Term numbers increased monotonically across all rounds with no resets, confirming zombie leader prevention worked correctly. Restarted nodes rejoined as Followers (confirmed via `/status` endpoint) and began accepting heartbeats from the new leader.

Conclusion: The system self-heals in ≈1.2 seconds on average with zero data loss. The availability gap during re-election is the correct trade-off for strong consistency.

5 Experiment 4: Geo-Distribution Tradeoff

5.1 Purpose and Tradeoff

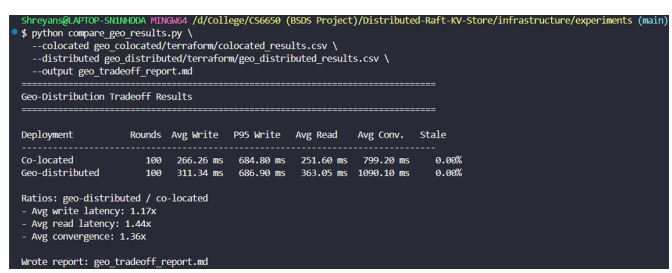
Question: How does spreading Raft nodes across AWS regions affect write latency, read latency, and replica convergence time compared to a co-located cluster?

Tradeoff: The main tradeoff is fault-domain isolation and potential read locality versus consensus latency. A geo-distributed cluster can survive a full regional outage and, in a real multi-region deployment, could serve reads from nearby replicas. However, Raft writes require communication with a majority of replicas, so cross-region RPCs are expected to increase write latency and slow replica convergence.

Setup: We compared two 3-node Raft deployments. In the co-located setup, all three nodes were deployed in `us-east-1` within the same VPC. In the geo-distributed setup, nodes were deployed across `us-east-1`, `us-east-2`, and `us-west-2`, using Elastic IPs for stable peer addressing across regions. Each setup ran for 100 sequential rounds. In each round, the script found the current leader, wrote a unique key to the leader, immediately read the key from all three replicas to check for stale reads, and then polled until all replicas returned the latest value to measure convergence time.

Limitation: The client running the experiment was located in `us-east-1`. Therefore, the experiment captures the cost of cross-region replication, but it does not fully capture the possible read-locality benefit of geo-distribution. Measuring that benefit would require deploying separate clients in each region. Also, the experiment used sequential rounds, so it measures request latency and convergence behavior rather than throughput under concurrent load.

5.2 Results



```
Shreyans@LAPTOP-SN1NHDOA MINGW64 /d/College/CS6650 (BSDS Project)/Distributed-Raft-KV-Store/infrastructure/experiments (main)
$ python compare_geo_results.py \
  --colocated geo_colocated/terraform/colocated_results.csv \
  --distributed geo_distributed/terraform/geo_distributed_results.csv \
  --output geo_tradeoff_report.md
================================================================================
Geo-Distribution Tradeoff Results
================================================================================

Deployment          Rounds  Avg Write   P95 Write   Avg Read    Avg Conv.   Stale
--------------------------------------------------------------------------------
Co-located             100   266.26 ms   684.80 ms   251.60 ms   799.20 ms    0.00%
Geo-distributed        100   311.34 ms   686.90 ms   363.05 ms  1090.10 ms    0.00%

Ratios: geo-distributed / co-located
- Avg write latency: 1.17x
- Avg read latency: 1.44x
- Avg convergence: 1.36x

Wrote report: geo_tradeoff_report.md
```

Figure 2: Geo Co-Located vs Geo Distributed Benchmarks

Metric	Co-located	Geo-dist.
Avg write latency	266 ms	311 ms
P95 write latency	685 ms	687 ms
Avg read latency	252 ms	363 ms
Avg convergence	799 ms	1090 ms
P95 convergence	1547 ms	1739 ms
Immediate stale reads	0/300	0/300

Table 4: Geo-distribution tradeoff results over 100 rounds per setup. Compared to co-location, the geo-distributed setup had 1.17× write latency, 1.44× read latency, and 1.36× convergence time. Raw CSV data is stored under `geo_colocated/terraform/` and `geo_distributed/terraform/`.

5.3 Analysis

The geo-distributed setup had higher average write latency than the co-located setup: 311 ms compared to 266 ms, or about **1.17**× slower. This matches the expected cost of cross-region Raft consensus, since the leader must replicate writes to a majority before committing. The increase was modest because the co-located setup already includes baseline overhead from Raft coordination, WAL persistence, and HTTP request handling. The P95 write latencies were nearly identical, 685 ms versus 687 ms, suggesting that tail latency in both deployments is influenced by occasional Raft coordination delays rather than only geographic distance.

Read latency was **1.44**× higher in the geo-distributed setup, increasing from 252 ms to 363 ms on average. This result is partly an artifact of the test setup: the client was located in `us-east-1`, so reads to replicas in `us-east-2` and `us-west-2` crossed regions. In a real geo-distributed system with clients located near each replica, this same placement could improve read latency by allowing clients to read from nearby followers. Our experiment therefore captures the cost of remote reads from one client location, but not the full benefit of regional read locality.

Replica convergence was also slower in the geo-distributed deployment. Average convergence time increased from 799 ms to 1090 ms, or about **1.36**× slower. This reflects the additional time needed for committed updates to become visible across remote followers. No immediate stale reads were observed in either run, with both setups reporting 0 stale reads out of 300 immediate replica reads. This indicates that, under this sequential workload, followers were able to apply committed updates quickly enough that stale reads were not observed.

Conclusion: In this experiment, the co-located setup performed better overall. It had lower write latency, lower read latency, and faster convergence. The geo-distributed setup introduced the expected cross-region coordination overhead, with writes 1.17× slower and convergence 1.36× slower. The observed read latency increase of 1.44× is mainly due to single-region client placement; a future experiment with clients deployed in each region would be needed to evaluate the read-locality benefit of geo-distribution more fairly.

6 Summary

Finding	Evidence
Async apply hides replication cost	Raft 57.5 ms < SimpleKVS 67.6 ms avg PUT
Disk I/O is the bottleneck	CPU 12.6% at saturation; snapshot flush every 100 writes
Sharding scales near-linearly	1.7× RPS gain at 2 shards
Zero data loss on failover	0 keys lost across all crash rounds
Geo cost modest for writes	266 → 311 ms avg (1.17×)

Table 5: Key findings across all four experiments.

Known system limitations: The Raft log is in-memory only; a full cluster restart loses uncommitted entries. Reads are stale by design (no follower consistency checks). The synchronous snapshot flush caps single-shard write throughput at ≈420 writes/sec before saturation. These are documented trade-offs, not bugs.